

1. Write a program to compute *commontail*[*u*, *v*], the longest common sublist of *u* and *v* ending with the ends of the lists. Thus

commontail[(A B C D E), (A C D E)] = (C D E).

2. Write a program to compute *mapleaf*[*f*, *x*], the list of expressions *f a* such that *a* is an atom occurring in *x*, appearing in the same order as the atoms appear in the printed expression. Thus

mapleaf[λ*z*.<*z*>, ((A . B) . C)] = ((A) (B) (C))

[Note that *mapleaf*[λ*z*.*z*, *x*]=*fringe x*]

3. Consider arithmetic expressions as represented in Chapters I and II. Namely an expression is

- (i) a number (satisfies *numberp*),
- (ii) a variable (not a number and satisfies *at*),
- (iii) a sum : PLUS . <list of expressions > or
- (iv) a product : TIMES . <list of expressions >.

(For simplicity, assume the sum and product lists always have at least 2 elements.)

The function *sop* converts such expressions into sum of products form, eg. the resulting expression is either a monomial or a sum of monomial terms which has the form PLUS . <list of monomials>. A monomial is either a number, a variable, or a product of the form TIMES . <list of numbers or variables >.

sop (TIMES (PLUS X 1) (PLUS Y Z) 3)=
 (PLUS (TIMES X Y 3) (TIMES X Z 3) (TIMES 1 Y 3) (TIMES 1 Z 3))

Write a program to compute *sop*. Try it on expressions returned by *diff*. What about *diff*[*sop e*, *v*] and *sop diff*[*sop e*, *v*]? Consider how you might convince a user of your program that *numval*[*e*, *alist*]=*numval*[*sop e*, *alist*]. (*numval* is defined on page 38.)

4. Consider an alternate representation of graphs by lists in which a graph is a list of edges and an edge is a list of the form

(<source vertex> <target vertex> . <data>).

and represents an edge of the graph from the <source vertex> to the <target vertex>. In the case of the simple unlabeled graphs described in Chapter I, the <data> is just NIL.

- 4.1. Write programs *mk-edge-rep*[*g*] and *mk-neigh-rep*[*g*] that convert a representation of a graph as a list of neighbors (a la Chapter I.) to a representation as a list of edges, and vice versa. Thus

$mk-edge-rep[((A\ B)(B\ A\ C\ D)(C\ D\ B)(D\ B\ C))]=$
 $((A\ B)(B\ A)(B\ C)(B\ D)(C\ D)(C\ B)(D\ B)(D\ C))$

$mk-neigh-rep[((A\ B)(B\ A)(B\ C)(B\ D)(C\ D)(C\ B)(D\ B)(D\ C))]=$
 $((A\ B)(B\ A\ C\ D)(C\ D\ B)(D\ B\ C))$

A current graph is a directed graph with edges labeled by numbers corresponding to the "current" flowing into the target vertex along that edge. A current graph satisfies Kirchoff's law if for each vertex, the sum of the currents flowing in is equal to the sum of the currents flowing out.

- 4.2. Write a program *Kirch*[*g*] that returns T if *g* is a current graph satisfying Kirchoff's law, and NIL otherwise. Thus

$Kirch[((A\ B\ 1)\ (A\ D\ 1)\ (B\ C\ 1)\ (C\ A\ 2)\ (D\ C\ 1))]=T$

$Kirch[((A\ D\ 1)\ (B\ D\ 1)\ (D\ C\ 2))]=NIL$

Continuing problem.

5. This problem deals with representation and manipulation of expressions in the pure λ -calculus. The basic properties and manipulations of LAMB-expressions are defined and you are asked to write the corresponding programs. The goal is a program that will apply a list of reductions to an expression.

The simplest λ -expression is a variable, represented by a LAMB-variable which is a pair $(X . n)$, $n=0, 1, 2, \dots$. The class, LAMB-expressions, of S-expressions representing λ -expressions is defined inductively as follows:

- (i) a LAMB-variable is a LAMB-expression (of sort variable),
- (ii) if e_1 and e_2 are LAMB-expressions then $(APPL\ e_1\ e_2)$ is a LAMB-expression (of sort application),
- (iii) if e is a LAMB-expression then for each n , $(LAMB\ n\ e)$ is a LAMB-expression (of sort abstraction), representing abstraction with respect to the n th variable $(X . n)$.
- (iv) These are all of the LAMB-expressions.

- 5.1. Write a program *islamb*[*e*] which returns T if *e* is a LAMB-expression and NIL otherwise.

[Remark: you will find programs computing the predicates *var*, *appl*, and *abstr* which are satisfied by the three types of LAMB-expressions useful, here and later.]

In the following we will use variable to mean LAMB-variable and expression to mean LAMB-expression.

An occurrence of a variable, v , in an expression e , can be classified as free or bound as follows. If e is the variable v , then the occurrence of v in e is free in e . If e is an abstraction on v , e.g. e is of the form $(LAMB\ n\ e_1)$ and v is $(X . n)$ then a free occurrence of v in e_1 is bound by this LAMB. If e is of the form $(LAMB\ n\ e_1)$, where v is not $(X . N)$, then an

occurrence of v (in $e1$) is bound or free in e according as it is bound or free in $e1$. Similarly, if e is of the form $(\text{APPL } e1 \ e2)$, an occurrence of v in $e1$ (resp. $e2$) is bound or free in e according as it is bound or free in $e1$ (resp. $e2$).

An occurrence of v in e is designated by the list of "moves", called a *location*, necessary to get from e to the subexpression of e which is the occurrence of v . If e is v then the location is the empty list. If e is $(\text{APPL } e1 \ e2)$ and the occurrence is in $e1$ then the location is $A1$ followed by the list of moves locating v in $e1$. (similarly if the occurrence is in $e2$, with $A2$ instead of $A1$.) If e is $(\text{LAMB } m \ e1)$ then the location is B followed by the list of moves locating v in $e1$.

- 5.2. Write a program *freeoccs* $[n, e]$ that returns a list of locations of free occurrences of the n th variable, $(X \ . \ n)$, in the expression e . (You may assume e to be a LAMB-expression.) Thus

freeoccs $[1, (\text{APPL } (\text{LAMB } 2 \ (\text{APPL } (X.2) \ (X.2))) \ (\text{APPL } (X.1) \ (X.1)))] = ((A2 \ A1)(A2 \ A2))$

- 5.3. Write a program *substfree* $[e1, n, e]$ which substitutes $e1$ for free occurrences of $(X \ . \ n)$ in e .

Note that we haven't provided a means to prevent trapping of free variables of $e1$. If $(X \ . \ m)$ occurs free in $e1$ and v occurs free in a subexpression of e of the form $(\text{LAMB } m \ e2)$ the occurrence of $(X \ . \ m)$ in $e1$ will be "trapped" upon substitution. That is it will be bound in the resulting expression. This causes undesired behaviour in the λ -calculus, so we wish to prevent such substitutions from occurring.

- 5.4. Write a program *freefor* $[e1, n, e]$ which returns T if no free variable of $e1$ will become bound upon substitution of $e1$ for each free occurrence of $(X \ . \ n)$ in e .

Since we don't want to be stuck just because some expression is not "freefor" we provide a means of modifying the expression e so that $e1$ will be free for v in e . This is known as renaming of bound variables. It is accomplished for a variable $(X \ . \ n)$, by choosing a variable $(X \ . \ m)$ that is unused so far and replacing each subexpression of e of the form $(\text{LAMB } n \ e2)$ by $(\text{LAMB } m \ e3)$ $e3$ is obtained from $e2$ by replacing each occurrence of $(X \ . \ n)$ in $e2$ bound by this LAMB by $(X \ . \ m)$.

- 5.5. Write a program *rename* $[n, m, e]$ that renames $(X \ . \ n)$ to $(X \ . \ m)$ in e as outlined above.

Now we have all of the basic programs needed to manipulate LAMB-expressions, carry out reductions, look for normal forms, etc. For example we can apply a list of reductions to an expression. There are two basic reductions. First, $(\text{ALPHA } n \ m)$ applied to e is *rename* $[n, m, e]$. It applies only if $(X \ . \ m)$ does not occur in e . Second, BETA applies only to expressions e of the form $(\text{APPL } e1 \ e2)$ where $e1$ is an abstraction $(\text{LAMB } n \ e3)$ and $e2$ is free for $(X \ . \ n)$ in $e1$. The result of applying BETA to e is *substfree* $[e2, n, e1]$. We generalize these reductions to apply to subexpressions of an expression e . Thus $(\text{ALPHA } n \ m \ loc)$ applied to e is obtained by replacing the subexpression located at *loc* by the result of applying $(\text{ALPHA } n \ m)$ to that subexpression. It only applies if *loc* is a valid location in e and the ALPHA reduction applies to that subexpression. Similarly for $(\text{BETA } loc)$.

- 5.6. Write a program *LAMB-reduce* $[e, rlist]$ that applies a list of reductions to a LAMB-expression. The program should be robust in the following sense. It should check that e is a LAMB-expression and return an appropriate message if not. It should check each element of *rlist* to

see that it denotes a reduction and complain if not. Before a reduction is applied, a check should be made that it does indeed apply and appropriate complaints made if not. Some examples follow. Let

```
e=(LAMB 0 (APPL (X . 1) (X . 0)))
e1=(LAMB 1 (APPL (X . 0) (X . 1)))
e2=(LAMB 2 (APPL (X . 2) (X . 1)))
e3=(LAMB 2 (APPL (X . 2) (LAMB 1 (APPL (X . 0) (X . 1)))))
e4=(APPL (LAMB 1 (LAMB 0 (APPL (X . 1) (X . 0))))
      (LAMB 1 (APPL (X . 0) (X . 1))))
rlist=((ALPHA 0 2 NIL) (BETA NIL))
```

then

```
freefor[e1, 1, e]=NIL
rename[0, 2, e]=e2
freefor[e1, 1, e2]=T
substfree[e1, 1, e2]=e3
LAMB-reduce[e4, rlist]=e3]
```

After we discuss sequential programs and I/O, we will see how *LAMB-reduce* can be converted in to an interactive program that you can have various conversations with about LAMB-expressions.